自己动手写操作系统

从零构建 x86 微内核

基于 SZY-KERNEL 项目实战

SZY

2026 年 3 月 16 日

本书随项目代码同步演进，每完成一个 Stage 即补充对应章节。

目录

I 准备工作 **9**

1 前言 **10**

1.1 这本书要做什么 10
1.2 目标读者 10
1.3 项目愿景 10
1.4 全书路线图 11
1.5 如何阅读本书 11

2 开发环境搭建 **12**

2.1 本章目标 12
2.2 宿主机环境 12
2.3 必要工具安装 12
2.4 项目目录结构 13
2.5 构建与运行闭环 13
2.6 VS Code 调试集成 13
2.7 本章小结 13

II 裸机基础设施 **14**

3 Multiboot2 引导与串口输出 **15**

3.1 本章目标 15
3.2 背景知识：x86 启动流程 15
3.2.1 Multiboot2 协议 15
3.2.2 实模式 → 保护模式 15
3.3 链接脚本 `linker.ld` 15
3.4 引导汇编 `boot.asm` 16
3.5 C 入口 `kmain` 16
3.6 串口驱动 `serial.c` 16
3.7 printk：内核的 printf 16

3.8 验证 16
3.9 本章小结 16

4 GDT：全局描述符表 17
4.1 本章目标 17
4.2 背景知识：段式内存管理 17
4.2.1 GDT 条目格式（8 字节） 17
4.2.2 段选择子 17
4.3 数据结构：`gdt.h` 与 `gdt.c` 17
4.4 刷新段寄存器：`gdt_flush.asm` 17
4.5 验证 17
4.6 本章小结 18

5 IDT：中断描述符表 19
5.1 本章目标 19
5.2 背景知识：中断与异常 19
5.2.1 IDT 条目格式 19
5.2.2 错误码 19
5.3 ISR 汇编 Stub：`isr_stubs.asm` 19
5.4 寄存器快照结构：`regs_t` 19
5.5 C 异常处理：`isr_handler_c` 19
5.6 验证：`int3` 自检 19
5.7 本章小结 20

6 IRQ、PIC 与键盘驱动 21
6.1 本章目标 21
6.2 背景知识：8259 PIC 21
6.2.1 IRQ 向量映射 21
6.3 PIC 初始化：`pic.c` 21
6.4 IRQ 分发框架：`irq.c` 21
6.5 IRQ 汇编 Stub：`irq_stubs.asm` 21
6.6 PS/2 键盘驱动：`keyboard.c` 21
6.7 验证 22
6.8 本章小结 22

7 交互式 Shell 23
7.1 本章目标 23
7.2 架构设计 23
7.3 Console 输入层：`console.c` 24

7.4 Shell 主循环：shell.c 24
7.5 命令接口：cmd.h 24
7.6 命令实现示例：cmd_cls.c 24
7.7 如何添加新命令 24
7.8 验证 24
7.9 本章小结 24

III 内存管理 25

8 PMM：物理内存管理 26
8.1 本章目标 26
8.2 背景知识：物理内存布局 27
8.3 可插拔后端架构 27
8.3.1 接口定义：pmm.h 27
8.3.2 Dispatch 层：pmm.c 27
8.4 Bitmap 分配器：pmm_bitmap.c 27
8.5 Buddy 分配器：pmm_buddy.c 27
8.6 后端切换：kconfig.h 27
8.7 Shell 命令 27
8.8 验证 27
8.9 本章小结 27

9 VMM：虚拟内存与分页 28
9.1 本章目标 28
9.2 背景知识：x86 二级分页 29
9.2.1 PDE / PTE 格式 29
9.2.2 TLB 与 invlpg 29
9.3 页目录与页表数据结构 29
9.4 核心 API：vmm_map_page 29
9.5 开启分页：vmm_init 29
9.6 Page Fault Handler 29
9.7 汇编辅助：paging_flush.asm 29
9.8 验证 29
9.9 本章小结 29

10 High-Half Kernel 30
10.1 本章目标 30
10.2 背景知识：High-Half Kernel 架构 30

10.3 链接脚本改动 31
10.4 Boot 阶段的临时映射 31
10.5 vmm_init 中的双重映射 31
10.6 拆除 Identity Mapping 31
10.7 验证 31
10.8 本章小结 31

11 内核堆：kmalloc 与 kfree 32
11.1 本章目标 32
11.2 背景知识：为什么需要内核堆 32
11.3 堆虚拟地址规划 32
11.4 可插拔后端架构 32
11.5 First-Fit 空闲链表：`heap_first_fit.c` 32
11.5.1 数据结构图示 32
11.6 Slab 分配器：`heap_slab.c` 32
11.7 kmalloc_init：初始映射 32
11.8 验证 32
11.9 本章小结 33

12 VMA：虚拟内存区域管理 34
12.1 本章目标 34
12.2 背景知识：为什么需要 VMA 34
12.2.1 Linux 的 VMA 34
12.3 可插拔后端架构 34
12.4 VMA 权限标志 34
12.5 红黑树后端：`vma_rbtree.c` 34
12.6 Page Fault 集成 34
12.7 Boot 区域注册 34
12.8 验证 34
12.9 本章小结 35

IV 进程与用户态 36

13 TSS 与 Ring 3：进入用户态 37
13.1 本章目标 37
13.2 背景知识：x86 特权级 37
13.2.1 TSS 的作用 37
13.2.2 从 Ring 0 到 Ring 3：`iret` 37

13.3 GDT 扩展 37
13.4 TSS 初始化 37
13.5 跳转到用户态 37
13.6 验证 38
13.7 本章小结 38

14 ELF 加载器 39
14.1 本章目标 39
14.2 背景知识：ELF 格式 39
14.2.1 ELF 加载流程 39
14.3 数据结构 39
14.4 实现详解 39
14.5 VMA 集成 39
14.6 验证 39
14.7 本章小结 39

15 系统调用 40
15.1 本章目标 40
15.2 背景知识：为什么需要系统调用 41
15.2.1 系统调用约定 41
15.3 IDT 配置 41
15.4 Syscall 分发 41
15.5 第一批系统调用 41
15.6 用户态封装 41
15.7 验证 41
15.8 本章小结 41

16 进程管理 42
16.1 本章目标 42
16.2 背景知识：进程抽象 42
16.3 PCB 数据结构 42
16.4 进程地址空间 42
16.5 fork：进程复制 42
16.6 exec：加载新程序 42
16.7 waitpid：等待子进程 42
16.8 验证 42
16.9 本章小结 42

17 调度器 **43**
17.1 本章目标 43
17.2 背景知识：CPU 调度 43
17.3 上下文切换 43
17.4 Round-Robin 调度器 44
17.5 PIT 定时器配置 44
17.6 验证 44
17.7 本章小结 44

V 文件系统 **45**

18 VFS：虚拟文件系统 **46**
18.1 本章目标 46
18.2 背景知识：为什么需要 VFS 46
18.3 VFS 数据结构 46
18.4 文件描述符表 46
18.5 系统调用集成 46
18.6 验证 46
18.7 本章小结 46

19 initrd / ramfs：内存文件系统 **47**
19.1 本章目标 47
19.2 背景知识：initrd 47
19.3 Ramfs 实现 47
19.4 GRUB Module 加载 47
19.5 验证 47
19.6 本章小结 47

VI 附录 **48**

A 工具链速查 **49**
A.1 构建命令 49
A.2 QEMU 常用选项 49
A.3 GDB 常用命令 49
A.4 Shell 命令速查 50

B x86 架构速查 **51**
B.1 寄存器 51
B.1.1 通用寄存器（32 位） 51
B.1.2 段寄存器 51
B.1.3 控制寄存器 51
B.2 GDT 条目格式 52
B.3 IDT 条目格式 52
B.4 PDE / PTE 格式 52
B.5 Page Fault 错误码 52
B.6 中断向量分配 53

Part I

准备工作

Chapter 1

前言

1.1 这本书要做什么

本书记录一个完整的 x86 微内核操作系统——SZY-KERNEL——从第一行汇编代码到能运行用户态程序的全过程。每一章对应项目的一个 Stage，包含：

- **目标**：这一步要实现什么功能
- **背景知识**：必要的硬件/CPU 原理
- **设计决策**：为什么选择这种方案而非其他
- **逐行代码讲解**：关键数据结构和实现
- **验证**：如何确认代码正确工作

1.2 目标读者

- 有 C 语言基础，了解指针、结构体、位运算
- 有基本的计算机组成原理概念（寄存器、内存、中断）
- 对”操作系统到底怎么工作的”感到好奇
- 不要求有汇编经验——书中会逐行解释每条汇编指令

1.3 项目愿景

1. 自举移植 GCC 编译器（在自己的 OS 上编译 C 代码）
2. 运行 Vim 文本编辑器

3. 支持 TCP/IP 网络（至少能 ping）

设计原则：

- **自研优先**——内存管理、分页、调度、系统调用全部自己写
- **POSIX 子集**——不追求完整，够移植 GCC/Vim 即可
- **微内核**——内核只做内存、调度、IPC，驱动尽量在用户态

1.4 全书路线图

Stage	章节	名称	里程碑
1	第 2 章	Multiboot2 引导与串口输出	A: 裸机基础
2	第 3 章	GDT：全局描述符表	A
3	第 4 章	IDT：中断描述符表	A
4	第 5 章	IRQ 与键盘驱动	A
5	第 6 章	交互式 Shell	A
6	第 7 章	PMM：物理内存管理	B: 内存管理
7	第 8 章	VMM：虚拟内存与分页	B
8	第 9 章	High-Half Kernel	C: 高级内存
9	第 10 章	内核堆 kmalloc	C
10	第 11 章	VMA：虚拟内存区域	C
11	第 12 章	TSS 与 Ring 3	D: 进程
12	第 13 章	ELF 加载器	D
13	第 14 章	系统调用	D
14	第 15 章	进程管理	D
15	第 16 章	调度器	D
16	第 17 章	VFS 虚拟文件系统	E: 文件系统
17	第 18 章	initrd / ramfs	E

1.5 如何阅读本书

建议按顺序阅读。每章末尾都有”本章小结”和”验证步骤”，确认当前 Stage 的代码能正常工作后再进入下一章。

所有代码可在 https://github.com/szylover/my_microkernel 找到，每个 Stage 对应一个 git tag 或 PR。

Chapter 2

开发环境搭建

2.1 本章目标

在动手写任何内核代码之前，先把工具链准备好：交叉编译器、汇编器、链接器、GRUB 打包工具和 QEMU 模拟器。

2.2 宿主机环境

- 操作系统：Ubuntu 22.04+ / WSL2（本书使用 WSL2 + VS Code Remote）
- 编辑器：VS Code + clangd 扩展

2.3 必要工具安装

```
sudo apt update
sudo apt install -y nasm gcc binutils grub-pc-bin \
    xorriso qemu-system-x86 gcc-multilib gdb bear
```

各工具的作用：

工具	用途
`nasm`	x86 汇编器（NASM 语法）
`gcc -m32`	C 编译器（32 位模式）
`ld`	链接器
`grub-mkrescue`	生成 GRUB 可引导 ISO
`xorriso`	ISO 9660 工具（grub-mkrescue 依赖）
`qemu-system-i386`	x86 模拟器
`gdb`	调试器

`bear`	生成 `compile_commands.json`（给 clangd）

2.4 项目目录结构

```
my_microkernel/
  src/
    boot/         # Multiboot2 引导 + 链接脚本
    include/      # 公共头文件
    kernel/       # 内核实现
  docs/           # 项目文档
  book/           # 本书 LaTeX 源码
  Makefile        # 顶层 wrapper
```

2.5 构建与运行闭环

```
cd src
make iso      # 编译内核 + 打包 ISO
make run      # QEMU 启动
make debug    # QEMU -S 等待 GDB 连接
make gdb      # 另一个终端启动 GDB
```

2.6 VS Code 调试集成

2.7 本章小结

工具链就绪后，下一章我们写第一行代码——Multiboot2 引导头和内核入口。

Part II

裸机基础设施

Chapter 3

Multiboot2 引导与串口输出

3.1 本章目标

- 编写 Multiboot2 兼容的引导头，让 GRUB 识别并加载我们的内核
- 从汇编跳转到 C 入口 `kmain`
- 通过 COM1 串口输出第一条调试信息
- 解析 Multiboot2 info 结构，打印 tag 列表

3.2 背景知识：x86 启动流程

3.2.1 Multiboot2 协议

3.2.2 实模式 → 保护模式

3.3 链接脚本 `linker.ld`

为什么？

为什么内核要加载到 1MiB？低于 1MiB 的地址空间被 BIOS、VGA 显存、ROM 等占据，不可靠。1MiB 是约定俗成的安全起点。

3.4 引导汇编 boot.asm

3.5 C 入口 kmain

3.6 串口驱动 serial.c

3.7 printk：内核的 printf

3.8 验证

```
make iso && make run
# 预期：终端（-serial stdio）看到
# [mb2] magic=36d76289
# [mb2] tag type=... size=...
```

3.9 本章小结

我们有了一个能被 GRUB 加载、跳转到 C 代码、并通过串口输出信息的最小内核。下一章我们设置 GDT，为后续的中断和内存管理做准备。

Chapter 4

GDT：全局描述符表

4.1 本章目标

- 理解 x86 保护模式下的段式内存管理
- 实现平坦内存模型的 GDT（Null + Kernel Code + Kernel Data）
- 用 `lgdt` 加载 GDT 并刷新段寄存器

4.2 背景知识：段式内存管理

4.2.1 GDT 条目格式（8 字节）

4.2.2 段选择子

4.3 数据结构：`gdt.h` 与 `gdt.c`

4.4 刷新段寄存器：`gdt_flush.asm`

CPU 状态变化

`lgdt` 后 CPU 开始使用新的 GDT。`jmp 0x08:flush` 将 CS 切换到新的内核代码段。随后 `mov ds/es/ss, 0x10` 切换数据段。此后所有内存访问都经过新 GDT 的段描述符。

4.5 验证

```
# 串口输出：
```

```
# [init] gdt ok
```

如果 GDT 有误（如 base/limit 计算错误），CPU 会 Triple Fault 并重启。

4.6 本章小结

GDT 在平坦模型下看似”多此一举”，但它是 x86 保护模式的硬性要求，也是后续 TSS（Ring 3 切换）的基础。下一章我们设置 IDT 来处理异常。

Chapter 5

IDT：中断描述符表

5.1 本章目标

- 理解 x86 中断与异常机制
- 实现 IDT，安装 CPU 异常 0–31 的处理程序
- ISR stub（汇编）→ C handler 的调用约定
- 用 `int3` 自检验证 IDT 正常工作

5.2 背景知识：中断与异常

5.2.1 IDT 条目格式

5.2.2 错误码

5.3 ISR 汇编 Stub：`isr_stubs.asm`

5.4 寄存器快照结构：`regs_t`

5.5 C 异常处理：`isr_handler_c`

5.6 验证：`int3` 自检

```
// kmain 中插入：
__asm__ volatile("int3");
// 预期：串口打印 #BP Breakpoint vector=3，然后正常返回继续执行
```

5.7 本章小结

IDT 是内核的” 事件分发表”。我们现在能捕获 CPU 异常并打印诊断信息，而不是直接 Triple Fault。下一章我们接入硬件中断——PIC 和键盘。

Chapter 6

IRQ、PIC 与键盘驱动

6.1 本章目标

- 理解 8259 PIC（可编程中断控制器）的工作原理
- Remap PIC 将 IRQ0–15 映射到 IDT 向量 0x20–0x2F
- 实现 IRQ 分发框架（注册 handler + EOI）
- 实现 PS/2 键盘驱动：scancode → ASCII + 环形缓冲区

6.2 背景知识：8259 PIC

6.2.1 IRQ 向量映射

6.3 PIC 初始化：`pic.c`

6.4 IRQ 分发框架：`irq.c`

6.5 IRQ 汇编 Stub：`irq_stubs.asm`

6.6 PS/2 键盘驱动：`keyboard.c`

设计决策

为什么用环形缓冲区？中断处理程序必须尽快返回（否则会阻塞其他中断）。把字符存入缓冲区、由 shell 主循环消费，是经典的”生产者-消费者”模式。

6.7 验证

```
make iso && make run
# 在 QEMU 窗口按键，串口应看到按键回显
```

6.8 本章小结

硬件中断链路打通：PIC → IRQ stub → C handler → 键盘驱动 → 缓冲区。下一章我们用这个输入做交互式 Shell。

Chapter 7

交互式 Shell

7.1 本章目标

- 实现 Console 输入聚合层（统一键盘 IRQ1 和串口轮询）
- 实现行编辑（回显、退格、回车）
- 实现命令注册表和分发机制
- 第一批命令：`cls`、`shutdown`、`cmds`

7.2 架构设计

设计决策

为什么抽象 Console 层？内核可能有多种输入源（键盘、串口、将来的网络终端）。Console 层统一提供 `console_getc()`，Shell 不关心输入从哪来。

7.3 Console 输入层：console.c

7.4 Shell 主循环：shell.c

7.5 命令接口：cmd.h

7.6 命令实现示例：cmd_cls.c

7.7 如何添加新命令

7.8 验证

```
szy-kernel > cmds
Commands (3):
cls  shutdown cmds
szy-kernel > cls
# 屏幕清空
```

7.9 本章小结

Shell 是内核开发阶段最重要的调试工具。有了可扩展的命令框架，后续每实现一个子系统（内存、进程等）都可以立即加入调试命令验证。

Part III

内存管理

Chapter 8

PMM：物理内存管理

8.1 本章目标

- 解析 Multiboot2 Memory Map，获取物理内存布局

- 实现可插拔 PMM 后端接口（`pmm_ops_t`）

- 第一个后端：bitmap 分配器

- 第二个后端：buddy system 分配器

- Shell 调试命令：`free`、`mmap`、`pmm`

8.2 背景知识：物理内存布局

8.3 可插拔后端架构

8.3.1 接口定义：pmm.h

8.3.2 Dispatch 层：pmm.c

8.4 Bitmap 分配器：pmm_bitmap.c

8.5 Buddy 分配器：pmm_buddy.c

8.6 后端切换：kconfig.h

8.7 Shell 命令

8.8 验证

```
szy-kernel > pmm state
PMM: base=0x00100000, total=32768, free=32395, used=373
PMM: backend=bitmap
szy-kernel > pmm alloc 3
pmm alloc: 0x07FC1000
pmm alloc: 0x07FC2000
pmm alloc: 0x07FC3000
szy-kernel > pmm freeall
pmm freeall: freed 3 pages
```

8.9 本章小结

PMM 是内存管理的基石——所有虚拟内存映射最终都需要物理页支撑。可插拔后端模式让我们能在不改调用方的前提下切换分配算法。下一章我们在 PMM 之上构建虚拟内存（分页）。

Chapter 9

VMM：虚拟内存与分页

9.1 本章目标

- 理解 x86 两级页表（Page Directory + Page Table）

- 实现 `vmm_map_page` / `vmm_unmap_page`

- 建立 Identity Mapping（虚拟地址 == 物理地址），开启分页

- 实现 Page Fault handler

- Shell 命令：`vmm state`、`vmm lookup`

9.2 背景知识：x86 二级分页

9.2.1 PDE / PTE 格式

9.2.2 TLB 与 `invlpg`

9.3 页目录与页表数据结构

9.4 核心 API：`vmm_map_page`

9.5 开启分页：`vmm_init`

CPU 状态变化

`vmm_load_page_directory(pd_phys)` 将 CR3 指向新页目录。此后所有内存访问都经过 MMU 翻译。如果新页目录没有覆盖当前 EIP 所在页 → Triple Fault。

9.6 Page Fault Handler

9.7 汇编辅助：`paging_flush.asm`

9.8 验证

```
szy-kernel > vmm state
VMM: paging enabled
VMM: 512 / 1024 PDE entries present
szy-kernel > vmm lookup 0xC0100000
virt 0xC0100000 -> phys 0x00100000 [P RW S]
szy-kernel > vmm fault
# 触发 Page Fault → 打印 CR2 + error code → halt
```

9.9 本章小结

分页开启后，内核的每一次内存访问都受 MMU 控制——这是内存保护和虚拟地址空间的基础。下一章我们把内核搬到高地址（High-Half Kernel）。

Chapter 10

High-Half Kernel

10.1 本章目标

- 将内核映射到虚拟地址 `0xC0000000`+（高半区）
- 拆除低地址 Identity Mapping，为用户态腾出 0–3GiB 空间
- 所有指针从物理地址切换到虚拟地址

10.2 背景知识：High-Half Kernel 架构

为什么？

为什么要 High-Half？让每个用户进程都有完整的 0–3GiB 虚拟地址空间，同时内核在所有进程的页表中都占据相同的高地址区域。切换进程时只需换低 3GiB 的映射，内核部分不变。

10.3 链接脚本改动

10.4 Boot 阶段的临时映射

10.5 vmm_init 中的双重映射

10.6 拆除 Identity Mapping

注意

拆除 identity mapping 前，必须确保所有指针（PMM bitmap、Multiboot2 info 等）都已转为虚拟地址（`PHYS_TO_VIRT`）。否则立即 Page Fault。

10.7 验证

```
szy-kernel > vmm lookup 0x00100000
virt 0x00100000 -> NOT MAPPED
szy-kernel > vmm lookup 0xC0100000
virt 0xC0100000 -> phys 0x00100000 [P RW S]
```

10.8 本章小结

内核现在安全地运行在高半区。低地址空间已清空，等待将来的用户态进程。下一章实现内核堆（kmalloc），让内核能做字节级的动态内存分配。

Chapter 11

内核堆：kmalloc 与 kfree

11.1 本章目标

- 在虚拟地址 `0xE0000000` 处建立内核堆区域
- 实现可插拔堆后端接口（`heap_ops_t`）
- 第一个后端：first-fit 空闲链表分配器
- 第二个后端：slab 分配器
- Shell 命令：`heap status`、`heap alloc`、`heap test`

11.2 背景知识：为什么需要内核堆

11.3 堆虚拟地址规划

11.4 可插拔后端架构

11.5 First-Fit 空闲链表：`heap_first_fit.c`

11.5.1 数据结构图示

11.6 Slab 分配器：`heap_slab.c`

11.7 kmalloc_init：初始映射

11.8 验证

```
szy-kernel > heap test
[heap-test] === kmalloc/kfree selftest ===
[heap-test] test 1: basic alloc/free ... PASS
[heap-test] test 2: multiple allocs ... no overlap: PASS
[heap-test] test 3: free all ... alloc_count == 0: PASS
[heap-test] test 4: reuse after free ... PASS
[heap-test] === ALL PASS ===
```

11.9 本章小结

有了 kmalloc/kfree，内核可以动态分配任意大小的内存。下一章实现 VMA（虚拟内存区域管理），让 Page Fault handler 能判断内存访问是否合法。

Chapter 12

VMA：虚拟内存区域管理

12.1 本章目标

- 理解 VMA 的作用：显式跟踪每个虚拟内存区域的地址范围和权限
- 实现可插拔 VMA 后端接口（`vma_ops_t`）
- 第一个后端：红黑树（经典 Linux 风格）
- Page Fault handler 集成 VMA 权限检查
- Shell 命令：`vma list`、`vma find`、`vma test`

12.2 背景知识：为什么需要 VMA

12.2.1 Linux 的 VMA

12.3 可插拔后端架构

12.4 VMA 权限标志

12.5 红黑树后端：`vma_rbtree.c`

12.6 Page Fault 集成

12.7 Boot 区域注册

12.8 验证

```
szy-kernel > vma list
VMA regions (2):
  [ 0] 0xC0000000 - 0xD0000000 256 MiB rwx direct-map
  [ 1] 0xE0000000 - 0xE0010000 64 KiB rw- kheap

szy-kernel > vma test
[vma-test] === VMA selftest ===
[vma-test] test 1: add + find ... PASS
[vma-test] test 2: find outside ... PASS
[vma-test] test 3: overlap detection ... PASS
[vma-test] test 4: remove + find ... PASS
[vma-test] === ALL PASS ===
```

12.9 本章小结

VMA 让内核从”隐式管理内存”进化到”显式跟踪每个区域”。这是实现按需分页(demand paging)、进程地址空间、mmap 的基础。内存管理子系统至此基本完成，下一部分进入进程与用户态。

Part IV

进程与用户态

Chapter 13

TSS 与 Ring 3：进入用户态

13.1 本章目标

- 理解 x86 特权级（Ring 0 – Ring 3）与特权级切换机制
- 实现 Task State Segment（TSS），配置内核栈指针
- 在 GDT 中添加 TSS 描述符和用户态代码/数据段
- 用 `iret` 从 Ring 0 跳转到 Ring 3 执行用户代码

13.2 背景知识：x86 特权级

13.2.1 TSS 的作用

13.2.2 从 Ring 0 到 Ring 3：`iret`

13.3 GDT 扩展

13.4 TSS 初始化

13.5 跳转到用户态

CPU 状态变化

`iret` 执行后：

- CPL 从 0 变为 3（用户态）
- CS 指向用户代码段，SS 指向用户数据段

- ESP 切换到用户态栈
- 内核页表中 PTE_USER 位控制哪些页用户态可访问

13.6 验证

13.7 本章小结

TSS + iret 让我们能在 Ring 0 和 Ring 3 之间切换。下一章加载 ELF 文件，让用户态执行真正的程序。

Chapter 14

ELF 加载器

14.1 本章目标

- 理解 ELF32 文件格式（ELF header、Program header、Section header）
- 实现 ELF 加载器：解析 ELF header → 加载 `.text`/`.data` 到用户地址空间
- 用 VMA 记录各段映射（代码段 r-x、数据段 rw-）

14.2 背景知识：ELF 格式

14.2.1 ELF 加载流程

14.3 数据结构

14.4 实现详解

14.5 VMA 集成

14.6 验证

14.7 本章小结

ELF 加载器让内核能运行外部编译的程序。下一章实现系统调用，让用户态程序能请求内核服务。

Chapter 15

系统调用

15.1 本章目标

- 理解系统调用的本质：用户态 → 内核态的受控入口

- 实现 `int 0x80` 系统调用门

- 第一批 syscall：`write`、`exit`、`brk`

- 用户态 syscall 封装库

15.2 背景知识：为什么需要系统调用

15.2.1 系统调用约定

15.3 IDT 配置

15.4 Syscall 分发

15.5 第一批系统调用

15.6 用户态封装

15.7 验证

15.8 本章小结

系统调用桥接了用户态和内核态。有了 write/exit，用户程序能输出信息和正常退出。下一章实现进程管理（PCB、fork、exec）。

Chapter 16

进程管理

16.1 本章目标

- 设计进程控制块（PCB / `task_struct`）
- 每个进程拥有独立的页表和 VMA 树
- 实现 `fork`（COW + VMA 复制）、`exec`、`waitpid`

16.2 背景知识：进程抽象

16.3 PCB 数据结构

16.4 进程地址空间

16.5 fork：进程复制

16.6 exec：加载新程序

16.7 waitpid：等待子进程

16.8 验证

16.9 本章小结

进程管理让内核能同时运行多个独立程序。下一章实现调度器，让这些进程分时共享 CPU。

Chapter 17

调度器

17.1 本章目标

- 实现时间片轮转（Round-Robin）调度
- 实现上下文切换：寄存器保存/恢复 + CR3 切换
- 利用 IRQ0（PIT 定时器）触发调度

17.2 背景知识：CPU 调度

17.3 上下文切换

CPU 状态变化

上下文切换后：

- CR3 指向新进程的页目录 → 地址空间切换
- TSS.ESP0 指向新进程的内核栈 → 中断时用正确的栈
- 通用寄存器恢复为新进程上次被切走时的状态

17.4 Round-Robin 调度器

17.5 PIT 定时器配置

17.6 验证

17.7 本章小结

调度器让多个进程分时运行。至此”进程与用户态”里程碑完成。下一部分进入文件系统。

Part V

文件系统

Chapter 18

VFS：虚拟文件系统

18.1 本章目标

- 设计 VFS 抽象层：统一 `open`/`read`/`write`/`close` 接口
- 实现文件描述符表（per-process fd table）
- 为后续具体文件系统（ramfs、FAT32）提供挂载框架

18.2 背景知识：为什么需要 VFS

18.3 VFS 数据结构

18.4 文件描述符表

18.5 系统调用集成

18.6 验证

18.7 本章小结

VFS 是文件系统的”可插拔接口”。下一章实现第一个具体文件系统 ramfs，让用户程序能读取打包在内核镜像中的文件。

Chapter 19

initrd / ramfs：内存文件系统

19.1 本章目标

- 实现 ramfs（内存中的简易文件系统）
- 将初始文件打包进内核镜像（initrd）
- ELF 加载器通过 VFS 读取文件，而非硬编码内存地址

19.2 背景知识：initrd

19.3 Ramfs 实现

19.4 GRUB Module 加载

19.5 验证

19.6 本章小结

有了 ramfs，用户程序可以通过标准文件系统接口访问文件。这为将来移植更复杂的文件系统（FAT32、ext2）和用户态工具链奠定基础。

Part VI

附录

附录 A

工具链速查

A.1 构建命令

命令	用途
`make iso`	编译内核 + 打包 GRUB ISO
`make run`	QEMU 启动 ISO
`make debug`	QEMU -S 等待 GDB（需 `DEBUG=1`）
`make gdb`	启动 GDB 并连接 QEMU
`make clean`	清理构建产物
`make compdb`	生成 `compile_commands.json`

A.2 QEMU 常用选项

选项	含义
`-serial stdio`	串口输出到终端
`-m 256`	分配 256MiB 内存
`-S`	启动时暂停 CPU
`-s`	开启 gdbstub（端口 1234）
`-no-reboot`	Triple Fault 时不自动重启
`-d int`	输出中断日志

A.3 GDB 常用命令

命令	用途
`b kmain`	在 kmain 设断点
`c`	继续运行
`n / s`	单步（不进入/进入函数）
`si / ni`	指令级单步
`info reg`	查看寄存器
`bt`	调用栈回溯
`x/10i $eip`	反汇编当前位置
`p/x var`	十六进制打印变量
`x/32wx $esp`	查看栈内存

A.4 Shell 命令速查

命令	用途
`cls`	清屏
`cmds`	列出所有命令
`free`	物理内存用量
`mmap`	Multiboot2 内存映射
`pmm state`	PMM 状态
`pmm alloc N`	分配 N 个物理页
`vmm state`	VMM 状态
`vmm lookup ADDR`	虚拟 → 物理地址翻译
`heap status`	堆统计
`heap test`	堆 selftest
`vma list`	列出所有 VMA
`vma find ADDR`	查找地址所属 VMA
`vma test`	VMA selftest
`shutdown`	关机

附录 B

x86 架构速查

B.1 寄存器

B.1.1 通用寄存器（32 位）

寄存器	惯用用途	调用约定
EAX	累加器、返回值	caller-saved
EBX	基址	callee-saved
ECX	计数器	caller-saved
EDX	数据、I/O	caller-saved
ESI	源索引	callee-saved
EDI	目的索引	callee-saved
EBP	栈帧基指针	callee-saved
ESP	栈指针	特殊

B.1.2 段寄存器

寄存器	用途
CS	代码段选择子（隐含 CPL）
DS	默认数据段
SS	栈段
ES	附加数据段
FS / GS	通用附加段（Linux 用 GS 做 per-CPU / TLS）

B.1.3 控制寄存器

寄存器	关键位
CR0	bit 0 (PE): 保护模式
	bit 31 (PG): 开启分页
CR2	Page Fault 时的故障虚拟地址
CR3	页目录物理地址
CR4	bit 4 (PSE): 4MiB 大页支持
	bit 7 (PGE): 全局页

B.2 GDT 条目格式

B.3 IDT 条目格式

B.4 PDE / PTE 格式

`bit 0` P (Present) —1= 有效，0= 访问触发 #PF

`bit 1` R/W —1= 读写，0= 只读

`bit 2` U/S —1= 用户态可访问，0= 仅内核

`bit 3` PWT —写透缓存策略

`bit 4` PCD —禁用缓存

`bit 5` A (Accessed) —CPU 访问后自动置 1

`bit 6` D (Dirty) —CPU 写入后自动置 1（仅 PTE）

`bit 7` PS/PAT —PDE: 4MiB 大页; PTE: PAT

`bit 8` G (Global) —TLB 不随 CR3 切换而刷新

`bit 9--11` AVL —软件可用位

`bit 12--31` 页帧物理地址高 20 位

B.5 Page Fault 错误码

`bit 0` P —0= 页不存在，1= 权限违规

`bit 1` W/R —0= 读操作，1= 写操作

`bit 2` U/S —0= 内核态，1= 用户态

`bit 3` RSVD —页表条目保留位被置位

bit 4 I/D —取指令引起

B.6 中断向量分配

向量	名称	说明
0	#DE	除零
3	#BP	断点（int3）
6	#UD	无效操作码
8	#DF	Double Fault
13	#GP	General Protection
14	#PF	Page Fault
32–47	IRQ0–15	硬件中断（PIC remap 后）
128	—	系统调用（int 0x80）